

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

**«Основы работы в среде Atmel Studio.
Архитектура и система команд микроконтроллера Atmega32»**

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Семенов П.О.

Санкт-Петербург
2026г.

1. Цель работы.

Изучение основ программирования микроконтроллеров семейства Atmel ATmega в среде разработки Atmel Studio. Знакомство с системой команд микроконтроллера ATmega32.

2. Теоретические сведения

2.1. Отладочная плата EasyAVR v7

Отладочная плата EasyAVR v7 является инструментальной платформой для разработки и отладки устройств на базе микроконтроллеров семейства Atmel AVR. Основные компоненты платы:

- 1) Встроенный программатор **mikroProg** для загрузки кода.
- 2) Четыре 8-разрядных порта ввода-вывода (PORTA, PORTB, PORTC, PORTD):
 - a) Каждый вывод имеет кнопку для ручного управления.
 - b) Каждый вывод имеет светодиод для визуализации состояния.
 - c) Выведены на три внешних разъёма для подключения периферии.
- 3) Интерфейсы связи:
 - a) **UART** с преобразователем FT232RL для связи через USB.
 - b) **I²C** шина с подключенной микросхемой EEPROM (1 КБ).
- 4) Встроенные устройства:
 - a) Четыре семисегментных индикатора.
 - b) Два потенциометра (имитация аналогового сигнала).
 - c) Динамик Piezo Buzzer (управление через ШИМ).
- 5) Разъёмы для расширения:
 - a) ЖК-дисплей (2x16 и 128x64).
 - b) Сенсорная панель.
 - c) Датчики температуры.
 - d) Модули GPS, GSM, Wi-Fi, microSD и другие.

2.2. Микроконтроллер Atmel AVR ATmega32

ATmega32 — 8-разрядный RISC-микроконтроллер с гарвардской архитектурой. Основные характеристики:

1) Архитектура и производительность:

- a) Гарвардская архитектура (раздельные шины для команд и данных).
- b) Большинство команд выполняется за 1 такт.
- c) 32 регистра общего назначения, напрямую подключенных к АЛУ.
- d) Производительность до 16 MIPS на частоте 16 МГц.

2) Память:

- a) 32 КБ Flash-памяти программ (с возможностью ISP-программирования).
- b) 1 КБ EEPROM (энергонезависимая память данных).
- c) 2 КБ SRAM (оперативная память).

d) Периферийные модули:

- a) 32 программируемые линии ввода/вывода.
- b) Таймеры/счётчики: два 8-разрядных и один 16-разрядный.
- c) 4 канала ШИМ (PWM).
- d) 8-канальный 10-битный АЦП.
- e) Интерфейсы: USART, SPI, I²C (TWI).
- f) Аналоговый компаратор.
- g) Сторожевой таймер (Watchdog).

2.3. Система команд и программирование

1) Адресное пространство:

- Раздельное для памяти программ (Flash) и данных (SRAM, регистры).

2) Регистр состояния SREG:

Бит	Обозначение	Название	Описание
0	C	Carry Flag	Флаг переноса
1	Z	Zero Flag	Флаг нулевого результата
2	N	Negative Flag	Флаг отрицательного результата
3	V	Overflow Flag	Флаг переполнения
4	S	Sign Flag	Бит знака ($S = N \oplus V$)
5	H	Half Carry Flag	Флаг половинного переноса
6	T	Transfer Bit	Пользовательский бит для операций с битами
7	I	Global Interrupt	Глобальное разрешение прерываний

Таблица 1. Назначение битов регистра SREG

1) Форматы команд:

- Используются различные способы адресации: прямая, относительная, косвенно-регистравая.
- Команды обработки данных, арифметические, логические, передачи управления.

2) Особенности программирования:

- **Бит T** регистра SREG используется командами BST (сохранить бит в T) и BLD (загрузить бит из T) для эффективной работы с отдельными битами.
- **Обработка многоразрядных чисел** осуществляется с помощью:
 - Команд для регистровых пар (ADIW, SBIW).
 - Команд с учётом флага переноса (ADC, SBC, ROR, ROL).
- **Средства ассемблера:** директивы `.def`, `.equ`, макросы для упрощения кода.

3) Формат HEX-файла:

- Используется для загрузки кода в микроконтроллер.
- Состоит из записей (строк), каждая содержит:
 - Маркер начала :
 - Длину данных (RECLen)
 - Адрес загрузки (LOAD OFFSET)

- Тип записи (RECTYP)
- Данные
- Контрольную сумму (CHKSUM)

3. Ход работы

В исходной программе был заменен фрагмент кода, отвечающий за функцию задержки, на код из варианта задания (см. Рис. 1).

```
delay:
    LDI    R30, 200; y
    LDI    R29, 153; x
delay_sub:
    INC    R29
    NOP
    BRNE   delay_sub
    NOP
    DEC    R30
    BRNE   delay_sub
    RET
```

Рис. 1. Код на языке ассемблера полученный из варианта

Далее был создан новый проект в Atmel Studio на языке ассемблера и размещен исходный код из п. 1 в файле проекта. В папке проекта был сформирован «.lss»-файл и «.hex»-файл после компиляции исходного кода, после чего была произведена запись «.hex»-файла в контроллер с помощью программы AVRFlash.

После была построена блок-схема исходного алгоритма (см. Рис. 2).

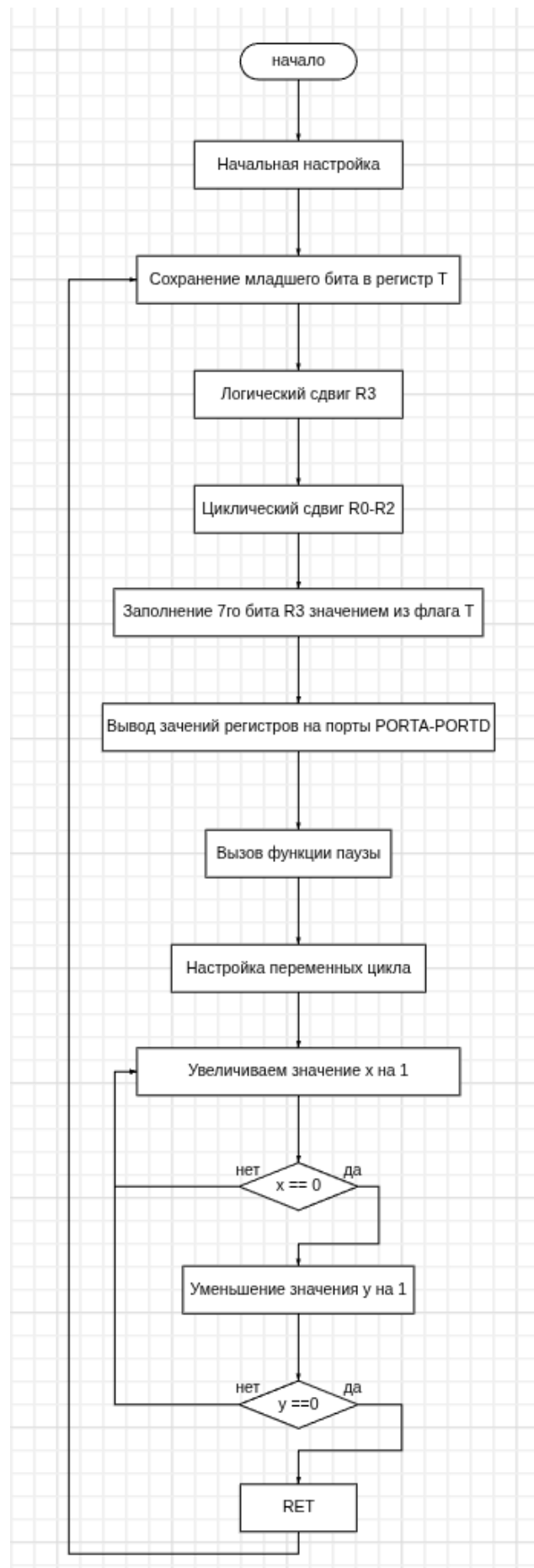


Рис. 2. Блок-схема кода циклического сдвига 32-битного числа

Далее было измерено время задержки в зависимости от переменных x и y , в данном коде это регистры R29 и R30 соответственно. Из блок-схемы на Рис. 2 видно, что функция задержки состоит из внутреннего и внешнего циклов.

Вначале производится $256 - x$ инкрементов над регистром R29, после чего программа переходит к внешнему циклу, уменьшает значение в регистре R30 на 1 и, если оно не равно нулю, переходит в начало внутреннего цикла, а так как значение в регистре $R29 = 0$, то при уменьшении этого регистра на один мы получим значение 0xFF, то есть 255 в десятичной системе счисления, а значит внутренний цикл должен пройти 255 раз перед тем, как значение в регистре R30 уменьшится на 1. Учтем, что микроконтроллер atmega32 работает на тактовой частоте 8 МГц, то есть выполняет 8 млн тактов в секунду.

Рассчитаем время работы функции-задержки в тактах:

$$\text{Кол-во тактов} = (1 + 1)_{\text{LDI}} + (1 + 1 + 1 + 1)_{\text{DEC+NOP+BRNE}} * (256 - x) - 1_{\text{BRNE}} + y * (256 * 4 + 4 - 1)_{\text{внешний цикл}} - 1_{\text{BRNE}} + 4_{\text{RET}}$$

Для значений $x = 153; y = 200$:

$$\text{Время задержки} = \frac{4 * (256 - 153) + y * (256 * 4 + 4 - 1) + 4}{8000000} = 25,727 \text{ мс}$$

После были рассчитаны параметры x и y так, чтобы функция-задержки выполнялась 0,01 секунды. Для этого нужно подобрать параметры так, чтобы $\frac{4 * (256 - x) + 1027y + 4}{8000000} = 0,01$. Были подобраны следующие значения: $x = 25, y = 77$, при этом в конец функции delay был добавлен один NOP для того, чтобы время задержки было ровно 0,01 секунда.

Далее было изучено содержимое .lss файла. В таблице 1 представлены метки программы и их адреса.

Название метки	Адрес в ПЗУП
delay:	000002
delay_sub:	000004
reset:	00000c
loop:	00001b

Таблица 2. Метки программы и их адрес в ПЗУП

Также были рассмотрены форматы команд в части состава и размерности операндов. (см. Таблица 3)

Операнды в команде	Примеры команд
1 × 16-битный адрес программы	JMP reset CALL delay BRNE delay_sub
1 × регистр (R0–R31)	DEC R12 INC R8 LSR R12 ROR R8
2 × регистра (R0–R31)	MOV R12, R8 ADD R12, R8 SUB R12, R8 EOR R12, R8
1 × регистр (R16–R31) + 8-битная константа	LDI R12, 0x10 LDI R13, 200 LDI TMP, 0x01
1 × регистр + битовый номер (0–7)	BST R12, 0 BLD R8, 7
1 × I/O-регистр + регистр (R0–R31)	OUT DDRA, R12 OUT DDRB, R8 OUT PORTA, R12 OUT PORTB, R8
Без операндов	NOP RET

Таблица 3. Форматы команд в части состава и размерности операндов

Было изучено содержимое .hex файла (см. Приложение 2). Первый байт в записи означает количество байт данных в записи, вторые 2 байта указывают на смещение, определяющее адрес загрузки данных, четвертый байт указывает на тип записи, далее идут данные и байт контрольной суммы.

Исходя из перечисленных выше данных можно определить количество записей в файле и количество машинных слов программы.

$$\text{Количество машинных слов} = \frac{0x10 + 0x10 + 0x10 + 0x10 + 0x0E}{2} = 39$$

$$\text{Количество записей} = 7$$

Рассмотрим порядок выполнения команды **ADD R12, R8** в центральном процессоре. Данная команда говорит процессору сложить значения регистров R12 и R8 и записать результат в регистр R12.

1) Этапы выполнения команды:

- Передача памяти программ (ПЗУП) адреса текущей команды.
- Извлечение команды из ПЗУП и запись в регистр команды: **ADD R12, R8**
- Декодирование команды – определение типа операции и формата.
- Извлечение адресов операндов (номеров регистров R12 и R8).
- Извлечение значений регистров R12 и R8 из регистрового файла.
- Передача операндов в АЛУ.
- Выполнение операции сложения: $R12 + R8$
- Запись флагов в SREG.
- Запись результата в регистр R12

2) Задействованные узлы микроконтроллера:

- Память программ (ПЗУП) — хранит машинный код команды.
- Счётчик команд (PC) — указывает адрес команды.
- Регистр команд — временное хранение команды при декодировании.
- Декодер команд — определяет тип операции и операнды.
- Регистровый файл — предоставляет доступ к регистра R12 и R8.
- АЛУ — выполняет операцию сложения.

3) Состав пересылаемых данных:

- По шине адреса: 16-битный адрес из PC в память программ.
- По шине данных: 16-битная команда из памяти в регистр команды.
- 8-битные значения регистров R12 и R8 на вход АЛУ.

- Новое значение R12 после сложения.

4) Состав управляющих сигналов:

- Выбрать регистр R12 в регистровом файле.
- Выбрать регистр R8 в регистровом файле.
- Передать значения регистров на вход АЛУ.
- Выполнить сложение.
- Записать результат в регистр R12.
- Отправить адрес из РС в ПЗУП.
- Записать значение из ПЗУП в регистр команд.

Далее были расшифрованы команды **CFFE** и **99E1** из варианта задания. Для начала команды были переведены в двоичную систему счисления.

$0xCFFE = 1100\ 1111\ 1111\ 1110$

$0x99E1 = 1001\ 1001\ 1110\ 0001$

1) Анализ инструкций:

- Для команды CFFE:
 - Шаблон $1100\ kkkk\ kkkk\ kkkk \rightarrow \mathbf{R JMP\ k}$:
 - Смещение: $k = -2$
 - Описание: безусловный относительный переход на предыдущую команду. -2
- Для команды 99E1:
 - Шаблон $1001\ 1001\ AAAA\ AAbb \rightarrow \mathbf{SBIC\ P, b}$
 - Порт и бит: $P = 28, b = 1$
 - Описание: пропуск следующей команды, если бит 1 порта 15 равен 0. EECR, 1

Данные команды были помещены в конец исходного кода, после программа была скомпилирована снова. В lss файле были найдены вставленные команды и проверен их код (см. Рис. 3). Таким образом была проверена правильность расшифровки.

```
metka:
000028 ba0b      OUT PORTA, R0
000029 ba18      OUT PORTB, R1
00002a cffe      RJMP metka
00002b 99e1      SBIC EECR, 1
```

Рис. 3. Исходные команды в .lss файле

4. Вывод

В ходе лабораторной работы были изучены основы программирования микроконтроллера ATmega32 в среде Atmel Studio, а также исследована архитектура и система команд данного микроконтроллера. Была успешно реализована программа с использованием ассемблера, проанализированы временные задержки и сформированы файлы прошивки. Полученные навыки позволяют эффективно разрабатывать и отлаживать программное обеспечение для микроконтроллеров семейства AVR.

5. Ответы на контрольные вопросы

1) Перечислите основные узлы микроконтроллера ATmega32 и укажите их назначение.

- Память программ (Flash-память, 32 КБ) – хранит исполняемый код программы.
- Оперативная память (SRAM, 2 КБ) – используется для временного хранения данных во время работы программы.
- EEPROM (1 КБ) – энергонезависимая память для хранения данных, сохраняющихся после отключения питания.
- Порты ввода-вывода (PORTA, PORTB, PORTC, PORTD) – обеспечивают взаимодействие с внешними устройствами (кнопки, светодиоды, дисплеи и др.).
- Два 8-разрядных таймеров-счётчиков с отдельными делителями частоты и режимами сравнения, используются для отсчёта времени.
- Аналого-цифровой преобразователь (АЦП, 10-битный, 8 каналов) – преобразует аналоговые сигналы (например, с датчиков) в цифровые данные.

- Последовательные интерфейсы (USART, SPI, I2C) – обеспечивают обмен данными с другими устройствами (например, ПК, датчиками, памятью).
- Аналоговый компаратор – сравнивает два аналоговых напряжения и формирует цифровой сигнал на основе результата сравнения.
- Watchdog-таймер – автоматически перезагружает микроконтроллер при зависании программы.
- Блок прерываний – обеспечивает реакцию на внешние и внутренние события без постоянного опроса.
- Осциллятор – задаёт темп работы микроконтроллера (кварцевый резонатор).
- Арифметико-логическое устройство - выполняет арифметические операции и логических сдвигов.

2) Укажите, в чём проявляются признаки RISC-архитектуры в микроконтроллере ATmega32. В чём преимущества и недостатки приведённых особенностей?

Признаки RISC-архитектуры:

- Большинство команд выполняется за один такт (кроме некоторых, например, вызовов подпрограмм, умножения).
- Фиксированная длина команд (16-битные, за редким исключением — 32-битные).
- Упрощённая система команд (131 инструкция), ориентированная на часто используемые операции.
- Регистровая архитектура — 32 регистра общего назначения, напрямую подключённых к АЛУ.
- Отдельные шины для памяти программ и данных.

Преимущества:

- Высокая скорость выполнения команд.
- Эффективное использование регистров снижает обращения к памяти

Недостатки:

- Сложные операции (умножение, деление) требуют нескольких команд.
- Фиксированная длина команд может приводить к менее плотному коду по сравнению с CISC
- Некоторые задачи требуют большего количества команд, чем в CISC-архитектурах.
- Усложненный конвейер и дешифрация команд.

3) От чего зависит время выполнения команд CPSE и BRHS? Приведите примеры кода (до 3-10 команд каждый), приводящие к различной продолжительности выполнения указанных команд.

Время выполнения данных команд зависит от того, будет ли выполнен переход и на сколько байт. Так, например, при выполнении команды CPSE если два передаваемых значения в регистрах равны и следующая команда занимает 1 машинное слово, то время выполнения 2 такта, если же 2 слова, то 3, при невыполнении условия 1 такт. Аналогично выявляется зависимость времени выполнения команды BRHS, если флаг полу-переноса установлен, выполняется 2 такта, в ином случае 1 такт.

Примеры кода, приводящие к различной продолжительности выполнения указанных команд:

- a) `LDI R29, 10`
`LDI R30, 10`
`CPSE R29, R30` - 2 такта
`NOP`
`NOP`
- b) `LDI R29, 12`
`LDI R30, 10`
`CPSE R29, R30` - 1 такт
`NOP`
`NOP`
- c) `LDI R29, 10`
`LDI R30, 10`
`CPSE R29, R30` - 3 такта
`JMP loop`
`NOP`

- 4) Приведите пример выполнения циклического сдвига вправо 8-разрядного числа (с переходом младшего разряда в старший) без использования флага Т. Приведите пример из трёх машинных команд, обеспечивающих сложение 24-разрядного числа с 24-разрядной константой.

Пример выполнения циклического сдвига вправо 8-разрядного числа:

```
MOV  R17, R16
LSL  R17
ROL  R16
```

Пример из трёх машинных команд, обеспечивающих сложение 24-разрядного числа с 24-разрядной константой:

```
SUBI  R18, 0xCD
SBCI  R17, 0xDD
SBCI  R16, 0xEE
```

- 5) Обоснуйте, чем вызваны ограничения допустимых значений номеров регистров и диапазонов констант в некоторых командах микроконтроллера ATmega32?

Данные ограничения связаны с тем, что команды atmega32 имеют фиксированный размер, большинство из которых имеют размер 16 бит. Если бы все регистры (0–31) и любые константы (0–255) были доступны во всех командах, потребовались бы дополнительные биты, что привело бы к увеличению длины команд (32 бита) или уменьшению количества доступных операций. И увеличением количества команд по сравнению с архитектурой RISC-16.

Приложения

Приложение 1

main.asm

```
; Объявляем переменную TMP для записи временных значений
.def TMP = R20

; Устанавливает адрес в памяти начиная с которого будет
размещаться код
.org $000
    JMP reset ; Указатель на начало программы

; Функция паузы
delay:
; Загрузка констант в регистры R30 и R29
    LDI R30, 200 ; y
    LDI R29, 153 ; x
delay_sub:
    INC R29 ; Увеличиваем на единицу значение регистра R29
    NOP ; Пустая команда, занимает один такт микроконтроллера,
выравнивает количество тактов, так как brne может выполняться за
один такт или за два, для более предсказуемого расчета времени
    BRNE delay_sub ; Переход на метку delay_sub если предыдущий
операнд не равен нулю
    NOP
    DEC R30 ; Уменьшаем на единицу значение регистра R30
    BRNE delay_sub
    NOP ; Для 10 миллисекунд нужен один пор чтобы было ровно
    RET ; Возврат из подпрограммы

; Начальная настройка
reset:
; Настройка исходных значений(можно сделать короче через просто
ldi R0 0x01 и можно было просто очистить все r а не через tmp
    LDI TMP, 0x01
    MOV R0, TMP ; Компирование значения регистра TMP в регистр R0
    CLR TMP ; Очистка всех битов регистра
    MOV R1, TMP
```

```

MOV R2, TMP
MOV R3, TMP
; Настройка портов ввода-вывода
SER TMP, 0xFF ; Установка всех битов регистра в значение 0xFF
OUT DDRA, TMP ; Вывод, запись значения регистра в порт
OUT DDRB, TMP ; Вывод
OUT DDRC, TMP ; Вывод
OUT DDRD, TMP ; Вывод
; Установка вершины стека в конец ОЗУ
LDI TMP, HIGH(RAMEND) ; Старшие разряды адреса, вычисляем
старший байт адреса RAMEND(последний адрес SRAM)
OUT SPH, TMP ; Загружаем в старший байт указателя стека
значение конца RAM
LDI TMP, LOW(RAMEND) ; Младшие разряды адреса, вычисляем
старший байт адреса RAMEND(последний адрес SRAM)
OUT SPL, TMP ; Загружаем в младший байт указателя стека
значение конца RAM

; Основной цикл
loop:
; Циклический сдвиг 32-разрядного числа R0-R3(если представить в
виде движения светодиода то он будет бегать с первой линиипотом
перепрыгнет из-за того, что )
BST R0, 0 ; Сохранение младшего бита во флаге T
LSR R3 ; Логический сдвиг вправо(здесь возможно есть ошибка и
нужно заменить LSR на ROR)
ROR R2 ; Циклический сдвиг вправо, если единица в последнем
бите, то при применении операции ROR последний бит перейдет в флаг
C(Carry) и при применении ROR на другом регистре он перенесет в 7
бит то, что было в флаге C
ROR R1 ; Циклический сдвиг вправо
ROR R0 ; Циклический сдвиг вправо
BLD R3, 7 ; Заполнение 7 бита значением из флага T

; Вывод 32-разрядного числа R0-R3 на порты PORTA-PORTD
OUT PORTA, R0
OUT PORTB, R1
OUT PORTC, R2
OUT PORTD, R3
; Пауза

```



```
CALL delay  
; Возврат в начало основного цикла  
RJMP loop ; Безусловный переход на метку
```

Приложение 2

main.hex

```
:0200000020000FC  
:100000000C940C00EDE4DAE1D3950000E9F7000070  
:10001000EA95D1F70000089541E0042E4427142EFC  
:10002000242E342E4FEF4ABB47BB44BB41BB48E0B4  
:100030004EBF4FE54DBF00FA3694279417940794AE  
:1000400037F80BBA18BA25BA32BA0E940200F3CFB9  
:000000001FF
```